VASIREDDY VENKATADRI INSTITUTE OF TECHNOLOGY
Permanently Affiliated to JNTU Kakinada, Approved by AICTE
Accredited by NAAC with 'A' Grade, ISO 9001:2008 Certified
Nambur, Pedakakani (M), Guntur (Dt) - 522508
Department of CSE - Artificial Intelligence & Machine Learning

MAJOR PROJECT REGISTRATION FORM

Date: 30 - 07 - 2026

Project Category:	☑ Research oriented ☐ Product oriented Specify if others: —	☐ Application oriented ☐ Other (please specify)	**Batch ID:**	AIML – B-14

Title of the Project: A Closed Loop Agentic Framework for Verifiable Code Translation

Abstract

Software codebase modernization, such as translating legacy systems from Python to modern, memory safe languages like Rust, is critical for achieving better application safety and execution performance. Although Large Language Models (LLMs) have shown impressive capabilities in automating code translation, relying on a single pass (one-shot) translation often results in syntactically invalid or logically incorrect code. This problem is magnified when target languages possess strict compilation rules, such as Rust's borrow checker. Since LLMs do not execute code during generation, they cannot independently verify the validity of their output, which leads to compilation failures and logical errors.

To overcome these limitations, this project presents an automated, closed loop program translation framework designed to enhance translation fidelity using sandboxed execution feedback. The system replaces single pass translation with an iterative, multi agent process. Initially, the translator agent produces target language code from the source script. Simultaneously, the framework mitigates "test translation bias"where an LLM writes biased tests that overlook its own translation bugs by generating unit tests and using differential testing. These tests are first executed on the source language to record correct behavior, and then translated to target language test cases. The translated code is compiled and executed against these tests inside a secure Docker container to prevent unsafe code execution. If compilation fails or tests do not pass, an error parser isolates compile time traces and assertion failures, presenting this clean diagnostic data back to the translation agent. The agent then runs a self correction pass, repeating the cycle until compilation and execution succeed.

We evaluate the framework using a collection of programming tasks translated from Python to Rust, comparing this iterative system against standard one shot LLM baselines. Performance is analyzed across compilation success rate, logical correctness (test pass rate), and average refinement iterations. The results are expected to show that compile and test feedback loops dramatically reduce the rate of syntax and logic failures, illustrating a highly reliable and practical methodology for automated, large scale software translation and verification.

MAJOR PROJECT REGISTRATION FORM

References (Category specific)

[1] X. Chen et al., "Teaching Large Language Models to Self Debug," arXiv preprint arXiv:2304.05128, 2023.

[2] A. Madaan et al., "Self Refine: Iterative Refinement with Self Feedback," in Advances in Neural Information Processing Systems (NeurIPS), vol. 36, 2023.

[3] H. Le, R. Wang, Y. Zhang, and S. C. H. Hoi, "CodeRL: Collaborating Code Generation and Program Evaluation with Deep Reinforcement Learning," in Advances in Neural Information Processing Systems (NeurIPS), 2022.

[4] B. Roziere, M. A. Lachaux, L. Chanussot, and G. Lample, "Unsupervised Translation of Programming Languages," in Advances in Neural Information Processing Systems (NeurIPS), 2020.

[5] A. Zhou, K. Yan, S. Sharda, and J. E. Gonzalez, "Language Agent Tree Search as a Generalist Policy for Code Generation," arXiv preprint arXiv:2310.04406, 2023.

[6] Y. Pan et al., "Automatically Translating Programs Between Languages: A Survey," arXiv preprint arXiv:2308.03310, 2023.

[7] C. Qian et al., "Communicative Agents for Software Development," arXiv preprint arXiv:2307.07924, 2023. (ChatDev: Multi agent framework for collaborative coding and testing)

[8] C. S. Xia and L. Zhang, "Keep the Conversation Going: Fixing Bugs with ChatGPT Using Conversational Automated Program Repair," arXiv preprint arXiv:2304.00385, 2023. (Conversational debugging and program repair loops)

[9] J. Zhang et al., "Self Edit: Fault Aware Code Editor for Code Generation," in Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL), 2023. (Iterative code editing using execution fault feedback)

[10] S. Lu et al., "CodeXGLUE: A Machine Learning Benchmark Dataset for Code Understanding and Generation," in Proceedings of the 30th International Joint Conference on Artificial Intelligence (IJCAI), 2021. (Foundational benchmark paper outlining code to code translation datasets)

[11] M. Chen et al., "Evaluating Large Language Models Trained on Code," arXiv preprint arXiv:2107.03374, 2021. (Foundational paper introducing the HumanEval dataset and unit test evaluation methodology)

[12] B. Roziere et al., "Leveraging Source to Source Translation for Code Security," arXiv preprint arXiv:2110.06778, 2021. (Investigates safety and compilation validation during programming language translation)

Project Team Members

S. No.	Regd. No.	Name of the Student	Signature
1	23BQ1A6189	Mothukuri Keerthi Medha	
2	23BQ1A6186	Medam Venkata Ajay Kumar Reddy	
3	23BQ1A6108	Karumuri Satya Sri	
4	23BQ1A6129	Korada Vamsi	

Project Guide

Name: Dr. D Murali Krishna Reddy Signature: